

Nightmare Navigator

Building a Horror Movie Telegram Bot in Go

Angela Scherer

Bärner Go X Elastic Talks 2026 no. 2 at Open Systems

Angela Scherer

Freelance Software / DevOps Engineer

Founder and CEO of Aiza GmbH

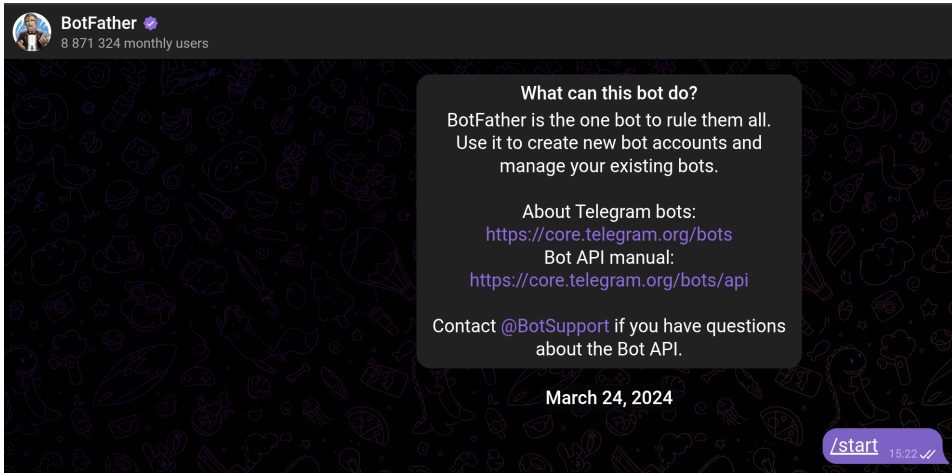
aiza.ch | github.com/angelaschereraiza

- Software Architecture and Engineering (Golang, C#, Python, JavaScript)
- Bug Fixing and Stabilization
- Kubernetes Engineering and Operations
- AI Infrastructure and Data Training
- Workshops and Training (Golang, Kubernetes, Podman Quadlet, AI)

1. Register the bot with Telegram
2. Choose the Go library
3. Parse flexible queries
4. Update recommendations on a schedule
5. Lessons learned

Flow in Telegram

1. Open @BotFather
2. Send /newbot
3. Choose display name
4. Choose username ending in bot
5. Copy the access token



I can help you create and manage Telegram bots.
If you're new to the Bot API, please [see the manual](#).

You can control me by sending these commands:

`/newbot` - create a new bot

`/mybots` - edit your bots

Edit Bots

`/setname` - change a bot's name

`/setdescription` - change bot description

`/setabouttext` - change bot about info

`/setuserpic` - change bot profile photo

`/setcommands` - change the list of commands

`/deletebot` - delete a bot

Bot Settings

`/token` - generate authorization token

`/revoke` - revoke bot access token

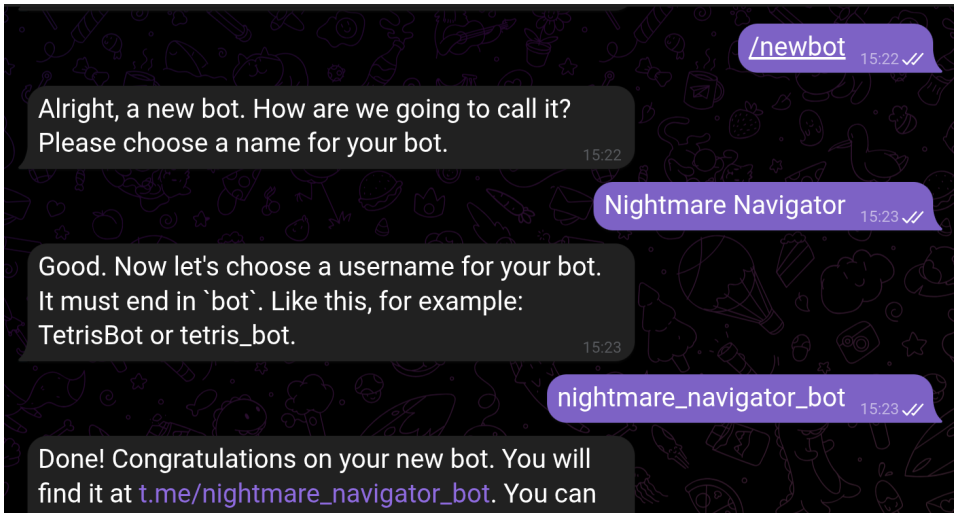
`/setinline` - toggle [inline mode](#)

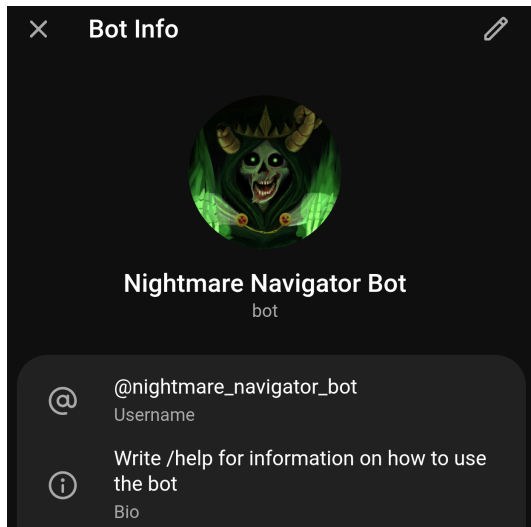
`/setinlinegeo` - toggle inline [location requests](#)

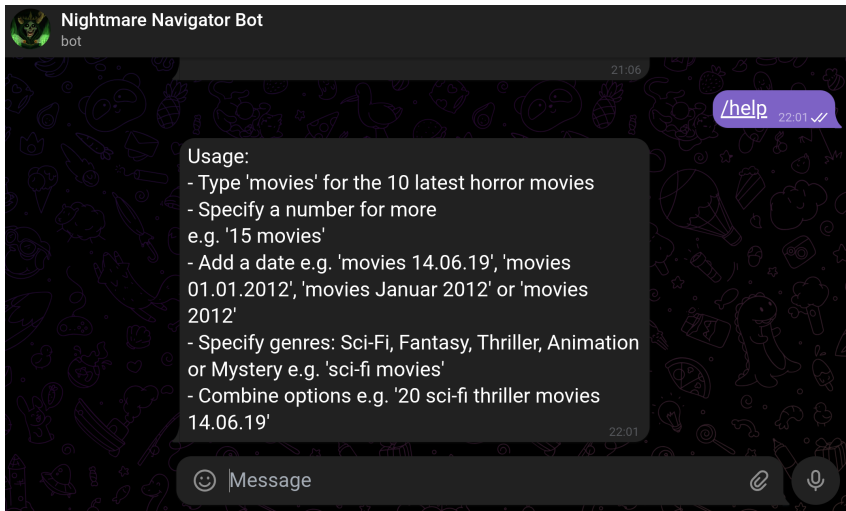
`/setinlinefeedback` - change [inline feedback](#) settings

`/setjoingroups` - can your bot be added to groups?

`/setprivacy` - toggle [privacy mode](#) in groups







There are **50+ Go libraries** for building Telegram bots.

The real choice is not popularity, but abstraction level:
lightweight wrapper or full framework?

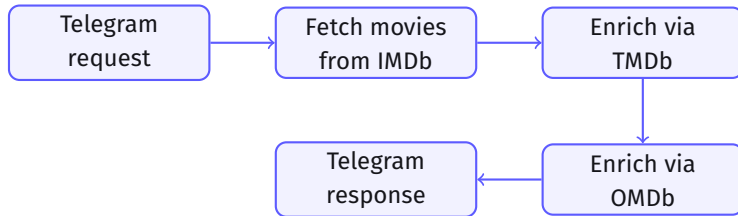
Library	Style / strengths	Stars
go-telegram-bot-api/v5	Minimal wrapper, close to Telegram API, widely adopted	6.4k
telebot	Framework-like handlers, middleware support, ergonomic API	4k
gotgbot	Generated API coverage, feature complete, strongly typed	1.5k
go-telegram/bot	Modern context-based API, lightweight abstraction	1k
telego	Clean modern design, strong typing, active development	800
OvyFlash/telegram-bot-api	Maintained fork with newer Telegram Bot API support	141

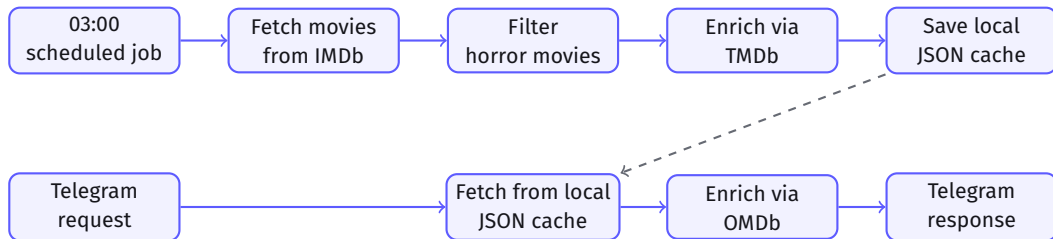
I used `github.com/OvyFlash/telegram-bot-api` because it stays very close to the official Telegram Bot API while still being lightweight and easy to use.

- Small wrapper around the Telegram Bot API
- Less framework abstraction and hidden behaviour
- Easy to understand during debugging
- Maintained fork with newer Telegram API support
- Good fit for a simple command-style bot

The bot combines multiple movie sources.

- **IMDb datasets:** ratings, vote counts, title IDs and genres
- **The Movie Database:** release dates, runtime and descriptions
- **OMDb:** country and age rating





```
bot, err := tgbotapi.NewBotAPI(cfg.TelegramBot.Token)
if err != nil {
    log.Panic(err)
}

u := tgbotapi.NewUpdate(0)
u.Timeout = 60

updates := bot.GetUpdatesChan(u)
```

Why I like this:

The code is explicit: receive updates, inspect the message, send a response.


```
cfg, err := config.LoadConfig("config.yaml")
if err != nil {
    log.Fatalf("failed to load config: %v", err)
}

imdbManager := imdb.NewIMDbManager(*cfg)
imdbManager.SaveLatestIMDbRatings()

bot.RunTelegramBot(cfg, imdbManager)
```

Startup does three things: load config, refresh movie data, then start listening for Telegram messages.

The bot accepts simple, human-ish Telegram messages.

- `movies`
- `15 movies`
- `sci-fi movies`
- `movies 2012`
- `20 sci-fi thriller movies 14.06.19`

```
GetFilteredMovieInfos(  
    util.ExtractCount(update.Message.Text),  
    util.ExtractGenres(update.Message.Text),  
    util.ExtractDate(update.Message.Text),  
    imdb.GetIMDbInfosByDateAndGenre,  
    BuildMovieInfoStrings,  
    *cfg,  
)
```

Instead of a complex command parser, the bot extracts three things: count, genres and date.

Dates are intentionally flexible.

- Numeric dates: 14.06.19, 14.06.2019
- Year only: 2012
- Month and year: Januar 2012
- Day, month and year: 01. Januar 2012

If no date is found, the bot uses today.

```
timer := time.NewTimer(durationUntilNextExecution())

go func() {
    for {
        <-timer.C
        executeAt0300AM(bot, cfg.TelegramBot.GroupId, *imdbManager, *cfg)
        timer.Reset(durationUntilNextExecution())
    }
}()
```

The bot refreshes its local movie data and can post newly found movies automatically to the group.

New movies are tracked in a small JSON file.

- Load already returned titles from disk
- Filter them out before sending new recommendations
- Save the updated list after sending
- Keep the behaviour stable across restarts

- A simple Telegram wrapper was enough for this project
- Most of the value is in data filtering and enrichment
- Local JSON cache keeps the bot fast and predictable
- Flexible input feels better than strict commands
- Scheduled refresh makes the bot useful without manual work

Nightmare Navigator in action



